UNIVERSIDAD PANAMERICANA

Ingeniería y ciencias aplicadas

Ingeniería en sistemas

Programación V – Base de datos II

Hipstagram

(P3 - Documento de diseño de arquitectura)

Kevin Eliazar Herrera Alburez (000113637)

Carlos Enrique Otoniel Bautista Sarazua (000133047)

Herbert David Estrada Trujillo (00141126)

Chimaltenango, Chimaltenango, abril 2026

Hipstagram

(P3 - Documento de diseño de arquitectura)

Kevin Eliazar Herrera Alburez (000113637)

Carlos Enrique Otoniel Bautista Sarazua (000133047)

Herbert David Estrada Trujillo (000141126)

Mario Obed Morales Güitz (Docente)

Chimaltenango, Chimaltenango, abril 2026

1 Introducción 1

2 Glosario 1

3 Diagrama de componentes 2

3.1 Figura 1. 2

3.2 Descripción de componentes 2

4 Vista lógica 3

4.1 Figura 2. 3

5 Diagrama de capas. 4

5.1 Figura 3. 4

6 Descripción de la seguridad y credenciales 5

7 Diagrama de seguridad 5

7.1 Figura 4. 5

8 Vista de la implementación 6

8.1 Figura 5. 6

8.2 Descripción general de la implementación 6

8.3 Figura 6. Diagrama de la implementación 7

9 Vista puesta en marcha 8

9.1 Figura 7. 8

10 Vista de datos 9

10.1 Figura 8. 9

11 Diagrama de entidad relación 10

11.1 Figura 9. 10

12 Diccionario de datos 11

12.1 Tabla roles 11

12.2 Tabla usuarios 11

12.3 Tabla publicaciones 11

12.4 Tabla Votos 11

12.5 Tabla LogsAuditoria 11

13 Definición de roles de la aplicación 12

13.1 Roles de interfaz y aplicación 12

13.2 Roles a nivel base de datos (Seguridad DBA en PostgreSQL) 12

14 Prototipo no funcional 13

14.1 Figura 10. Interfaz PC 13

15 Alcance y limitaciones de la aplicación 14

15.1 Alcance 14

15.2 Limitaciones 14

1 Introducción

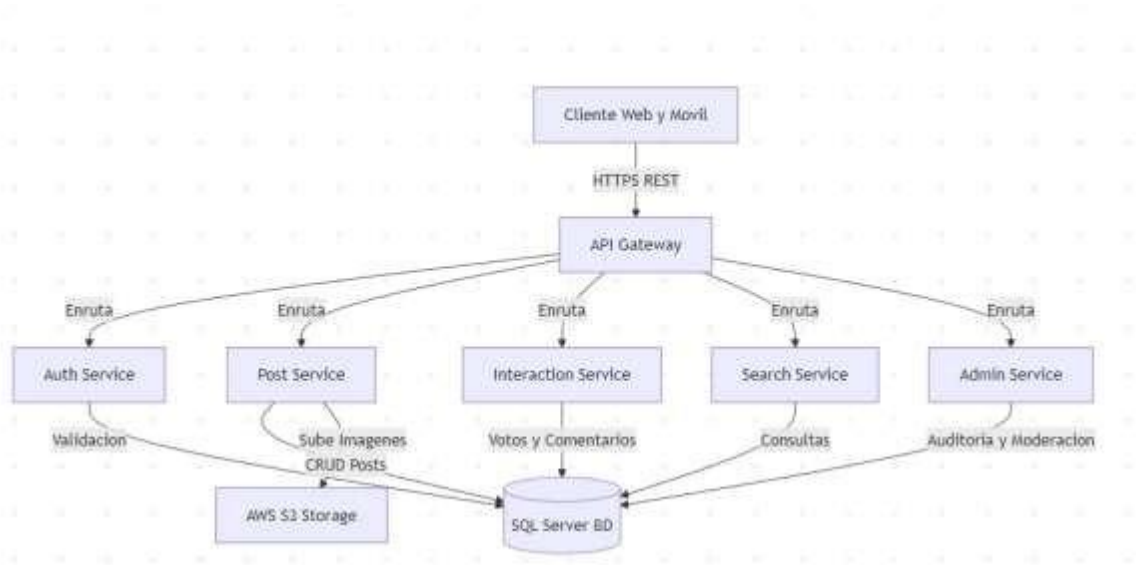
Hipstagram representa una solución avanzada en el ámbito de las redes sociales, fundamentada en una arquitectura de microservicios distribuida. Este diseño permite la independencia operativa de funciones críticas como la seguridad, la gestión de contenido y la moderación automatizada. El presente documento de diseño arquitectónico detalla la infraestructura lógica y técnica, asegurando la escalabilidad y la alta disponibilidad mediante el uso de contenedores y servicios de orquestación en la nube.

2 Glosario

Arquitectura Distribuida	Sistema cuyos componentes se encuentran en diferentes plataformas de red y se comunican mediante paso de mensajes para lograr un objetivo común.
Backend for Frontend:	Patrón de diseño donde se crean microservicios específicos para satisfacer las necesidades de interfaces de usuario particulares.
Disponibilidad	Protocolo que asegura que el sistema permanezca operativo y accesible durante un porcentaje de tiempo extremadamente alto, minimizando interrupciones.
Escalabilidad	Capacidad del sistema para manejar un crecimiento en la carga de trabajo (usuarios o datos) mediante la adición de recursos sin perder rendimiento.
Endpoint	Punto de conexión o URL específica de un microservicio donde se reciben las peticiones de los clientes.
Infraestructura como Código	Gestión y aprovisionamiento de la infraestructura tecnológica a través de archivos de configuración en lugar de procesos manuales.
Latencia	El tiempo de demora que toma un paquete de datos en viajar de un punto a otro dentro de la arquitectura de red.
Middleware	Capa de software que se sitúa entre el sistema operativo y las aplicaciones para facilitar la comunicación y gestión de datos entre microservicios.
Persistencia de Datos	Característica de los datos que permite que estos sobrevivan tras la finalización del proceso que los creó, almacenándolos de forma no volátil (Bases de Datos).
Amazon S3	Servicio de almacenamiento de objetos diseñado para almacenar y recuperar cualquier cantidad de datos.
VPC	Red virtual aislada lógicamente en la nube de AWS donde se despliegan los recursos del sistema para mayor seguridad.
AWS Fargate	Motor de computación para contenedores que permite ejecutar Docker sin tener que gestionar servidores físicos.
Subred Privada	Segmento de red que no tiene acceso directo desde internet, utilizado para proteger la base de datos y la lógica de negocio.
API Gateway	Punto de entrada único que gestiona las solicitudes de los clientes, se encarga del enrutamiento, la autenticación y el control de tráfico.
Microservicios	Estilo arquitectónico que estructura una aplicación como un conjunto de servicios pequeños, autónomos y acoplados de forma laxa.
JWT	Estándar abierto para transmitir información de forma segura entre partes como un objeto JSON, utilizado para la autenticación de usuarios.
Persistencia Híbrida	Estrategia que combina diferentes métodos de almacenamiento para optimizar el rendimiento.
Integridad referencial	Regla de base de datos que garantiza que las relaciones entre tablas se mantengan constantes.

3 Diagrama de componentes

3.1 Figura 1.



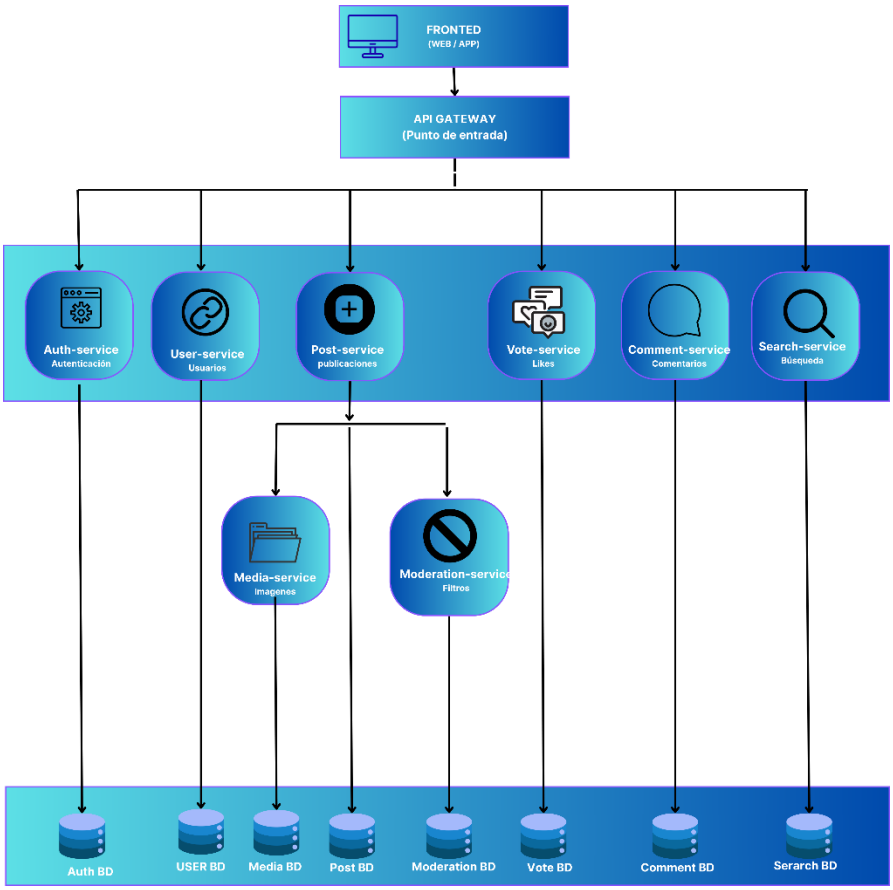
3.2 Descripción de componentes

- Cliente Web y Móvil: Las interfaces de usuario final (desarrolladas posiblemente en React o Flutter) que consumen las APIs del sistema.
- API Gateway: Es el punto de entrada único al sistema. Se encarga de recibir las peticiones HTTP, validar la firma de los tokens JWT y enrutar el tráfico hacia el microservicio correspondiente.
- Auth Service: Microservicio encargado de la seguridad. Gestiona el registro, validación de credenciales (hashing), inicio de sesión y emisión de los tokens JWT con los roles de usuario.
- Post Service: Encargado de la lógica de publicaciones. Valida la longitud de la descripción (máximo 128 caracteres), extrae hashtags y guarda el archivo binario de la foto en la nube.
- Interaction Service: Gestiona la lógica de las interacciones sociales, específicamente los comentarios paginados y el sistema de votos (garantizando un único voto por usuario por publicación).
- Search Service: Componente optimizado para búsquedas. Permite encontrar contenido mediante filtrado por hashtags y búsqueda por texto libre, además de generar el feed ordenado por "likes".
- Base de Datos (PostgreSQL): Almacenamiento relacional centralizado, estructurado con estrictas reglas de integridad referencial y seguridad (roles de base de datos).

4 Vista lógica

Hipstagram presenta una arquitectura de microservicios desacoplados donde el API Gateway actúa como orquestador central. Se observa la especialización de componentes como el Auth-service para la seguridad y el Post-service para la gestión de contenido, este último apoyado por servicios de moderación y procesamiento de medios. Cada módulo posee su propia persistencia de datos independiente, asegurando que fallos en un servicio no comprometan la disponibilidad global del sistema.

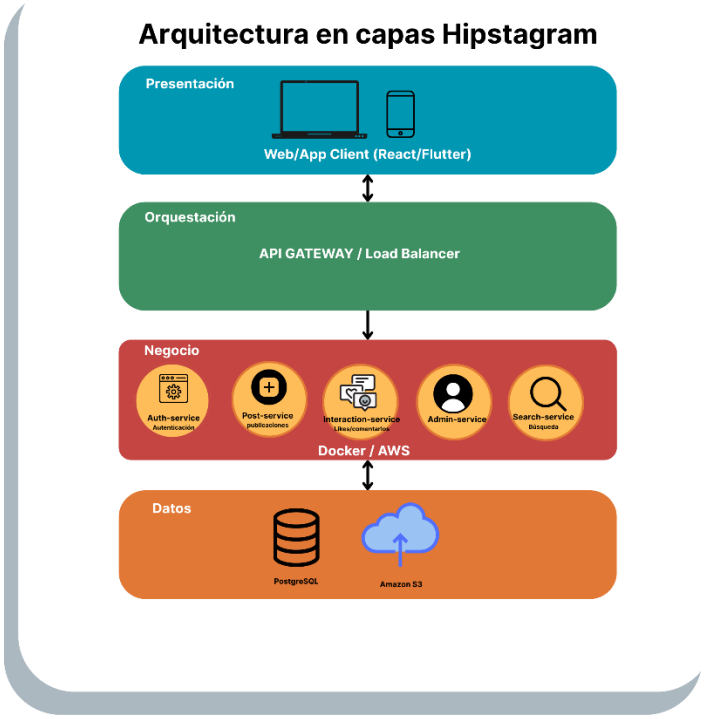
4.1 Figura 2.



5 Diagrama de capas.

La arquitectura de Hipstagram se ha diseñado bajo un modelo de n-capas desacopladas, lo que permite la escalabilidad independiente de cada componente y establece perímetros de seguridad claros entre el usuario y los datos sensibles.

5.1 Figura 3.



- Capa de presentación: Representa el punto de interacción con el cliente final mediante aplicaciones desarrolladas para web y móvil, su función principal es la captura de eventes, la renderización de la interfaz y el consumo de servicios externos a través de protocolos HTTPS.
- Capa de orquestación: Actúa como un proxy inverso y punto de entrado al ecosistema de microservicios, este nivel se centraliza en la seguridad mediante la validación de firmas JWT y la gestión de permisos.
- Capa de negocios: Contiene la lógica operativa del sistema, segmentada en servicios autónomos, cada servicio se ejecuta de forma aislada dentro de contenedores Docker, garantizando que el consumo de recursos sea eficiente y escalable según la demanda.
- Capa de datos: El nivel más profundo de la arquitectura, donde se garantiza la durabilidad de la información. PostgreSQL será nuestro motor relacional que gestiona la consistencia transaccional y la integridad referencial. Amazon S3 se refiere como tal a AWS, aquí es donde se resguardan físicamente los archivos multimedia.

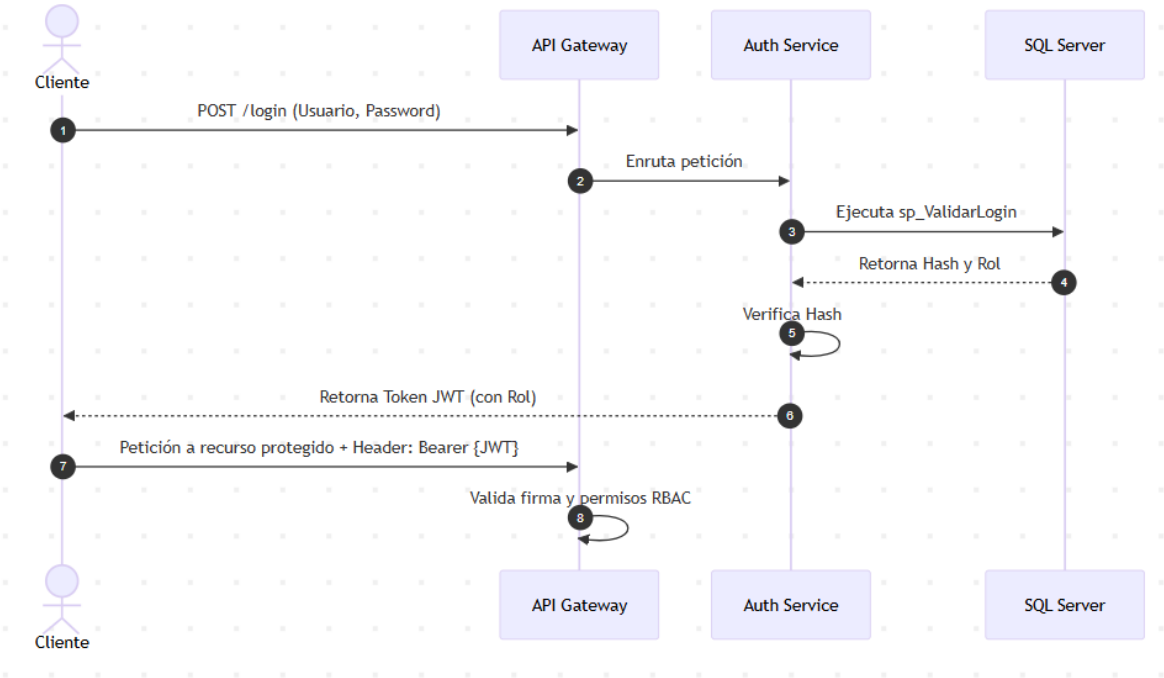
6 Descripción de la seguridad y credenciales

La seguridad del sistema Hipstagram se fundamenta en un modelo robusto de Autenticación y Autorización (RBAC).

- Autenticación: Las credenciales de los usuarios no se almacenan en texto plano; se utiliza un algoritmo de hashing fuerte (BCrypt/Argon2) en el backend (C#). Una vez validadas las credenciales contra PostgreSQL, el sistema emite un JSON Web Token (JWT) firmado digitalmente y un Refresh Token de corta duración.
- Autorización: El JWT contiene los claims del rol del usuario (Admin o Usuario). Cada microservicio valida la firma del token antes de procesar una petición, garantizando que los usuarios regulares no puedan acceder a endpoints administrativos (Error 403 Forbidden).
- Gestión de Secretos: Las cadenas de conexión a la base de datos y las llaves secretas del JWT nunca se versionan en código fuente. Se inyectan en los contenedores a través de variables de entorno seguras gestionadas por AWS y el pipeline de CI/CD.

7 Diagrama de seguridad

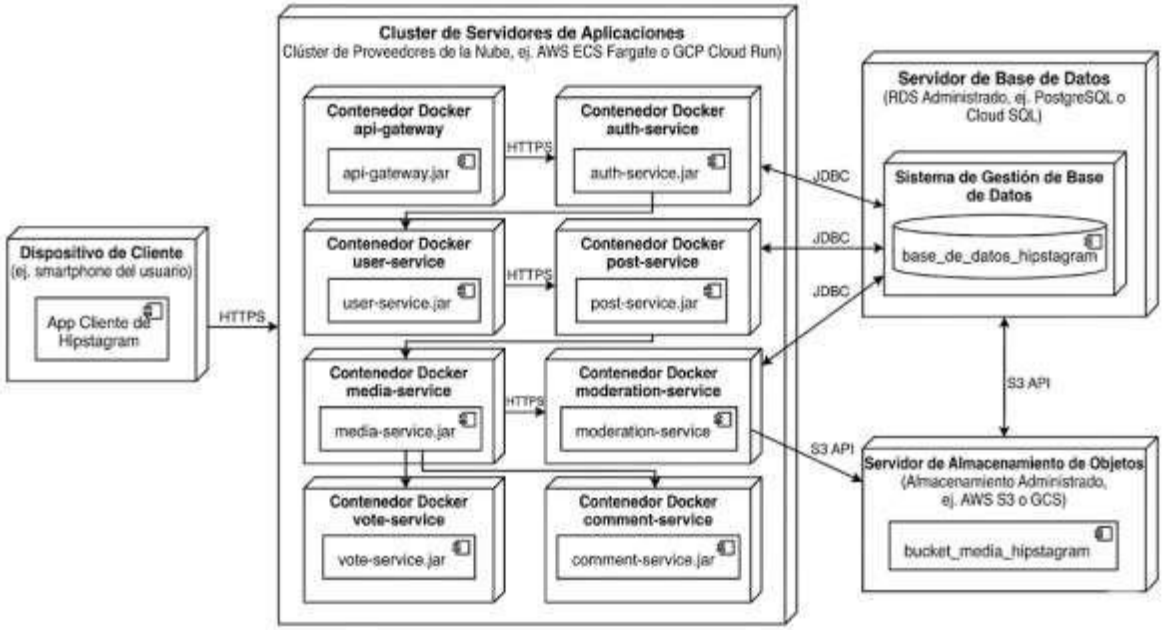
7.1 Figura 4.



8 Vista de la implementación

El sistema está diseñado bajo una arquitectura de microservicios contenedorizados. El código fuente de cada microservicio se desarrollará utilizando C# con .NET (Visual Studio 2026). Cada componente tendrá su propio archivo Dockerfile basado en Linux. Para el entorno de desarrollo local, todos los servicios, junto con el motor de base de datos PostgreSQL, se orquestarán de forma conjunta utilizando docker-compose. El control de versiones se gestionará en GitHub bajo la metodología estricta de GitFlow (ramas main, develop, feature, release y hotfix).

8.1 Figura 5.



8.2 Descripción general de la implementación

- Dispositivo de Cliente: Este sería la computadora o el teléfono que usaría el usuario que estaría usando la aplicación.
- Servidor de Aplicaciones: Es un clúster gestionado que administra los contenedores de Docker de los microservicios.
- Servidor de Base de Datos: Un servicio de base de datos administrado que garantiza alta disponibilidad y copias de seguridad de los esquemas de datos.
- Servidor de almacenamiento de Objetos: Un servicio especializado para almacenar físicamente las fotografías.

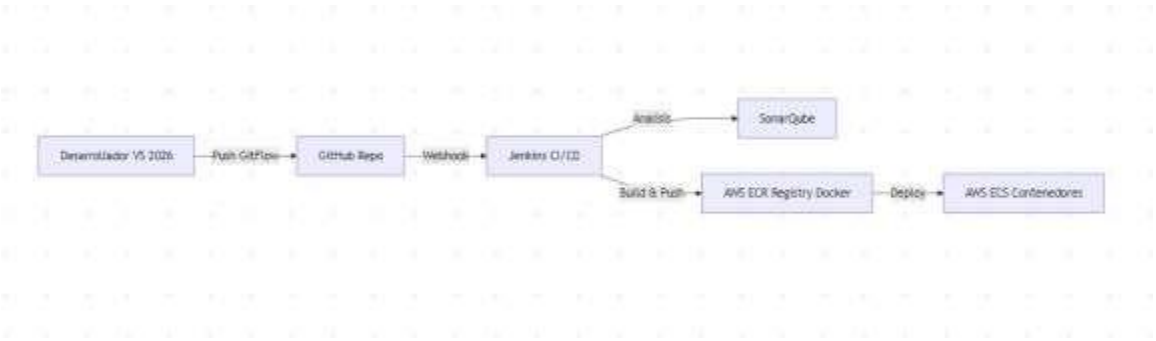
Protocolos y Conexiones:

- HTTPS: Protocolo seguro para toda la comunicación entre el cliente, el API y los microservicios.
- JDBC: Protocolo estándar para que los microservicios ejecuten consultas en PostgreSQL, pero en el servidor de base de datos.
- API S3: Protocolo para ejecutar el almacenamiento de datos.

Artefactos y Contenedores:

- App Cliente de “Hipstagram”: Parte del Frontend en la que interactúan directamente los usuarios.
- Contenedor Docker: Cada microservicio autónomo está empaquetado en su propia imagen de Docker para un aislamiento total.

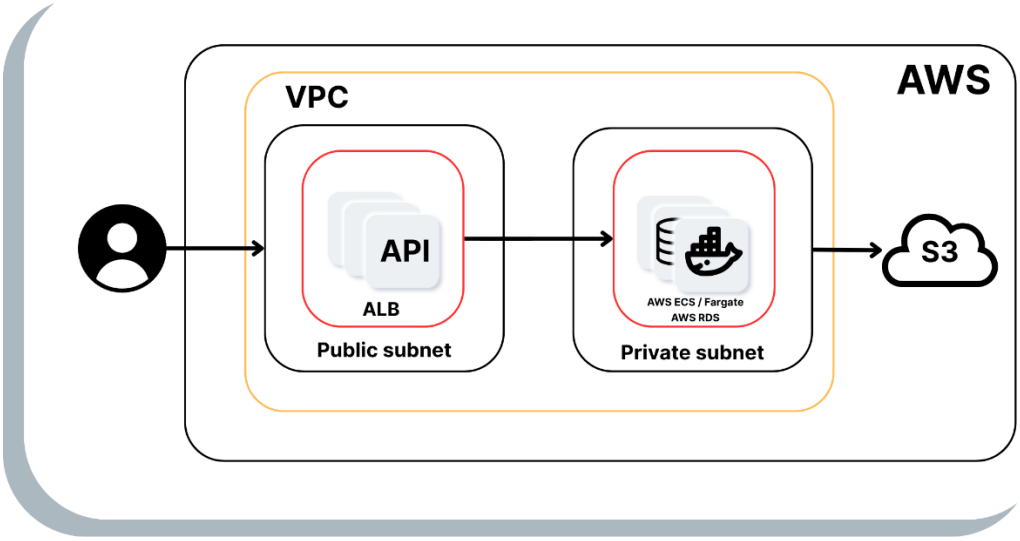
8.3 Figura 6. Diagrama de la implementación



9 Vista puesta en marcha

La infraestructura se despliega sobre una VPC segmentada para maximizar la seguridad, el flujo comienza cuando el usuario interactúa con la Aplication Load Balancer en la subred pública, este orquestador redirige el tráfico hacia los microservicios alojados en la subred privada, los cuales se ejecutan mediante AWS ECS y RDS y externamente hacia Amazon S3 para el almacenamiento multimedia, asegurando una arquitectura elástica y alternamente disponible.

9.1 Figura 7.

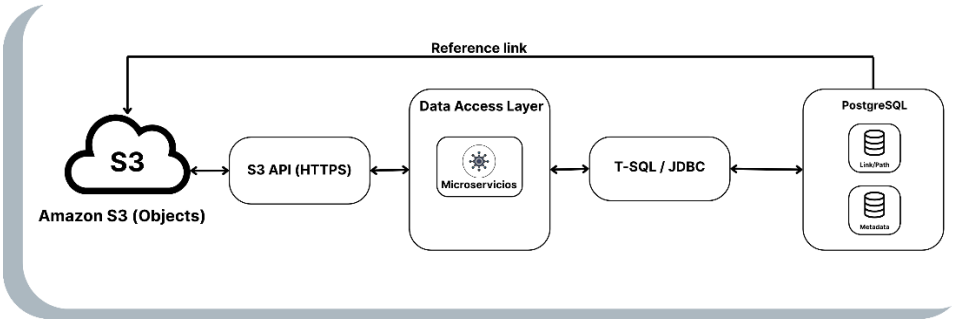


- El punto de entrada será un Application Load Balancer (ALB) conectado al API Gateway.
- Los microservicios en se ejecutarán en AWS ECS (Fargate), garantizando escalabilidad automática sin necesidad de gestionar servidores físicos.
- La persistencia de datos relacional estará alojada en AWS RDS (PostgreSQL), configurada con respaldos automáticos y Multi-AZ para alta disponibilidad.
- Los archivos multimedia (las fotografías publicadas por los usuarios) se almacenarán de forma directa y segura en Amazon S3.

10 Vista de datos

La vista de datos de Hipstagram define cómo se gestiona, fluye y almacena la información, asegurando la integridad referencial y la alta disponibilidad de los activos digitales, Se ha implementado una estrategia de persistencia híbrida para separar la lógica relacional de los datos binarios pesados.

10.1 Figura 8.



PostgreSQL:

- Integridad y normalización: La base de datos sigues las reglas de normalización para eliminar redundancias, las relacionales entre usuarios, publicaciones, likes y comentarios están protegidas mediante llaves foráneas (FK).
- Reglas de borrado: Se utiliza la restricción ON DELETE NO ACTION en tablas críticas, para evitar que la eliminación de un perfil borre accidentalmente registros de auditoria necesarios para el sistema.
- Transaccionalidad: Todas las operaciones que involucran múltiples tablas se ejecutan bajo transacciones atómicas, si un paso fallo, el sistema realiza un rollback automático para evitar datos inconsistentes.

Amazon S3:

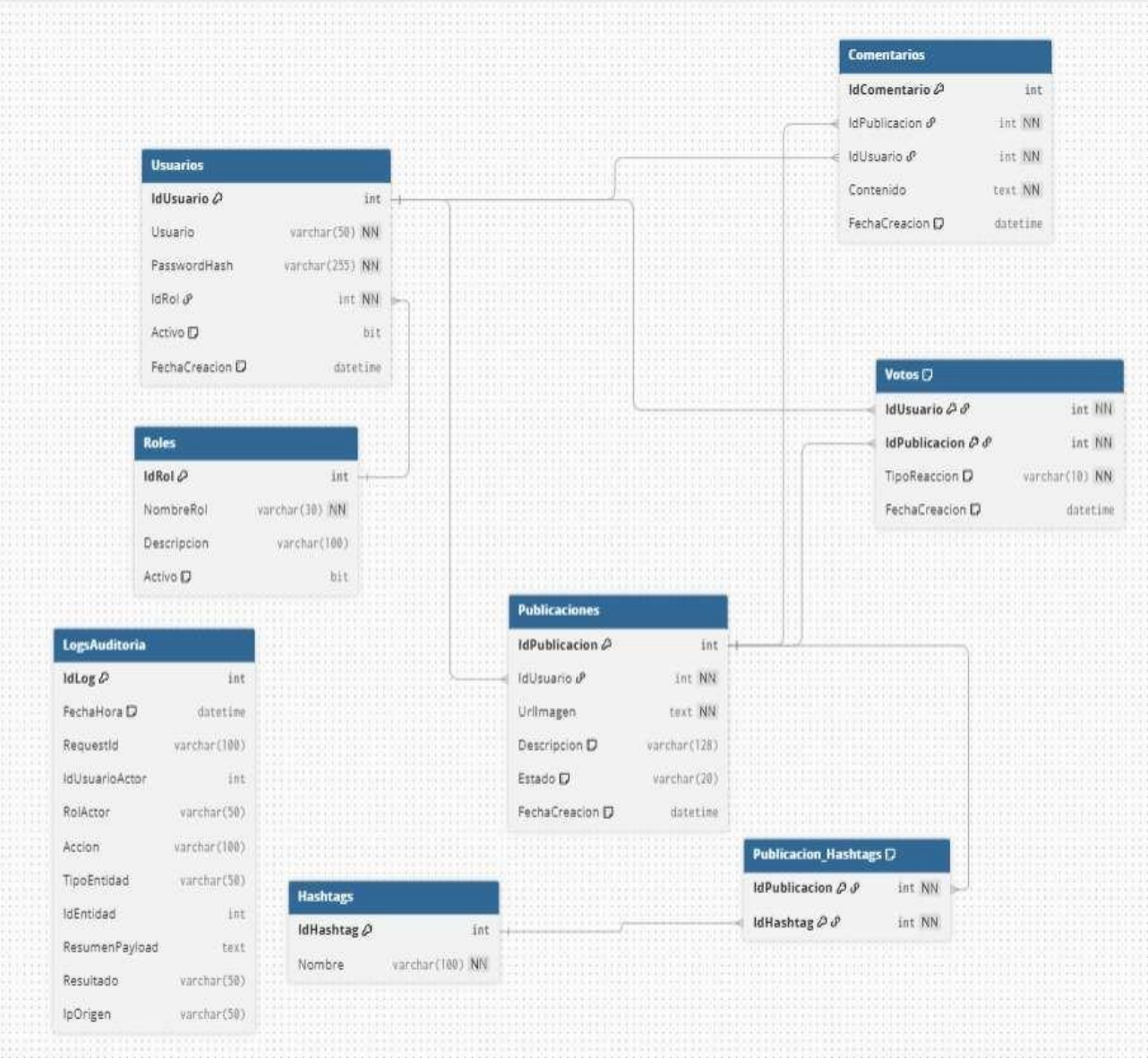
- Referencia lógica: PostgreSQL no guarda la imagen, sino un URL único que apunta al objeto en el bucket S3.
- Desacoplamiento: Esto permite que la base de datos se mantenga ligera y rápida, mientras que el almacenamiento de archivos puede crecer de forma limitada sin afectar los tiempos de respuestas de las consultas.

Flujo de persistencia:

- Recepción: El microservicio de contenido recibe el archivo y los metadatos.
- Escritura en S3: El archivo se sube al bucket y se obtiene una URL de confirmación.
- Registro: Tras la subida exitosa a S3, el microservicio inserta en la base de datos el registro con la descripción, hashtags y el link al archivo.
- Confirmación El sistema devuelve un estado éxitos al usuario.

11 Diagrama de entidad relación

11.1 Figura 9.



12 Diccionario de datos

12.1 Tabla roles

- IdRol (INT, PK): Identificador único autoincremental del rol.
- NombreRol (NVARCHAR 30, UNIQUE, NOT NULL): Nombre del rol (Admin, Usuario).
- Descripcion (NVARCHAR 100): Detalles de los permisos del rol.
- Activo (BIT): Bandera lógica de habilitación (1=Activo, 0=Inactivo).

12.2 Tabla usuarios

- IdUsuario (INT, PK): Identificador único del usuario.
- Usuario (NVARCHAR 50, UNIQUE, NOT NULL): Nombre de usuario para login.
- PasswordHash (NVARCHAR 255, NOT NULL): Contraseña encriptada.
- IdRol (INT, FK, NOT NULL): Llave foránea hacia la tabla Roles.
- Activo (BIT): Bandera lógica para bloqueo de usuarios.
- FechaCreacion (DATETIME): Fecha de registro en el sistema.

12.3 Tabla publicaciones

- IdPublicacion (INT, PK): ID único de la foto. IdUsuario (INT, FK, NOT NULL): ID del autor.
- UrlImagen (TEXT, NOT NULL): Ruta del archivo almacenado en AWS S3. Descripcion (VARCHAR 128): Texto corto de la publicación.
- Estado (VARCHAR 20): Estado de moderación (OK o BLOQUEADO).

12.4 Tabla Votos

- IdUsuario (INT, PK/FK): ID del usuario que vota.
- IdPublicacion (INT, PK/FK): ID del post votado. (Forma Llave Primaria Compuesta).
- TipoReaccion (VARCHAR 10): Tipo de voto (LIKE o DISLIKE).

12.5 Tabla LogsAuditoria

- Tabla aislada (sin FKs para garantizar inmutabilidad) que registra campos como: FechaHora, IdUsuarioActor, Accion, IdEntidad, Resultado e IpOrigen. Visible únicamente por administradores.

13 Definición de roles de la aplicación

El sistema se divide en dos niveles de seguridad en cuanto a roles:

13.1 Roles de interfaz y aplicación

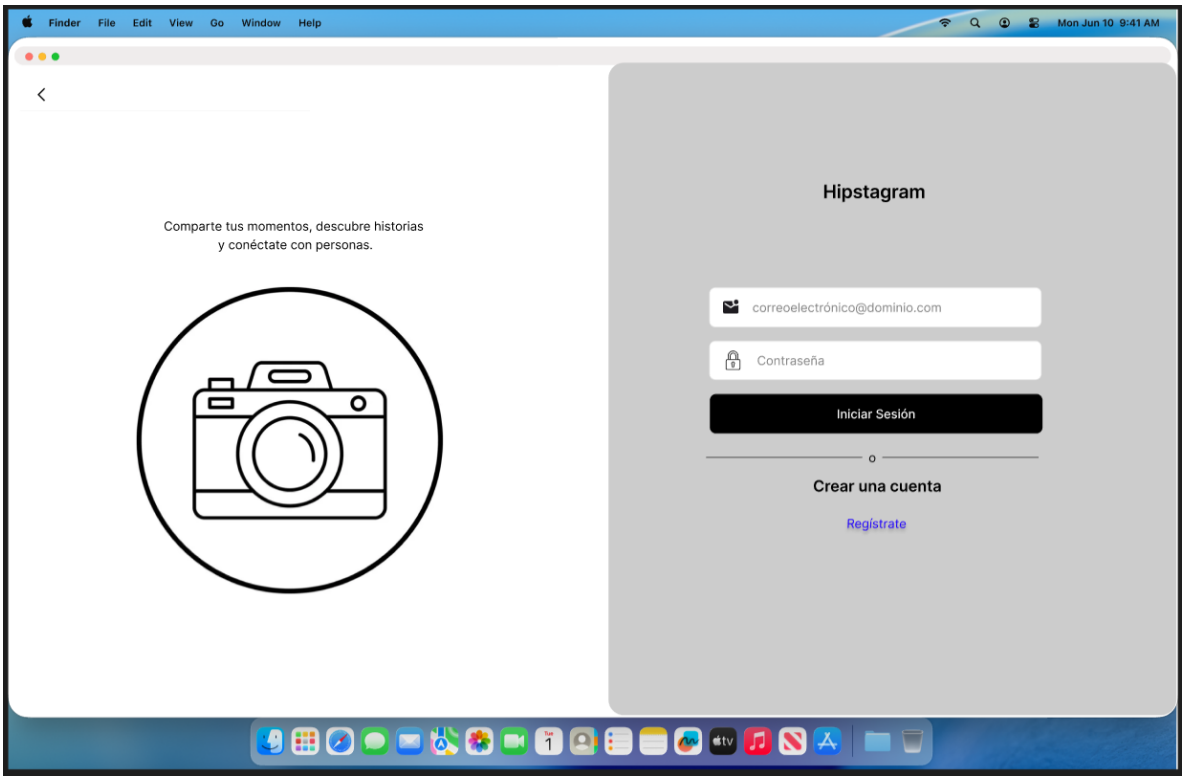
- USER: Usuario regular. Tiene permisos para consultar el feed, registrarse, publicar fotos, votar, comentar y buscar. Puede borrar exclusivamente sus propias publicaciones.
- ADMIN: Administrador de la plataforma. Posee acceso al Panel Admin para moderar publicaciones (bloquear contenido inapropiado), auditar usuarios, modificar el JSON de palabras restringidas y visualizar el Log de Auditoría.

13.2 Roles a nivel base de datos (Seguridad DBA en PostgreSQL)

- dba_role: Rol con privilegios DDL para realizar migraciones y cambios de estructura.
- app_role: Rol de uso exclusivo de los microservicios. Tiene permisos DML (SELECT, INSERT, UPDATE) sobre tablas operativas, pero tiene denegado el permiso de lectura sobre LogsAuditoria.
- admin_read_role: Rol creado para el microservicio de administración con permisos explícitos para consultar el registro de auditoría.

14 Prototipo no funcional

14.1 Figura 10. Interfaz PC



Descripción: El prototipo de la interfaz adaptada para computadora, este diseño representa una ventana en computadora para visualizar como se vería en un escritorio, el sistema estará adaptado para que funcione en web y apps.

15 Alcance y limitaciones de la aplicación

15.1 Alcance

El sistema está diseñado para soportar una operación concurrente basada en dos perfiles de acceso definidos: USER y ADMIN. El alcance integra los siguientes pilares:

- Funcional: Gestión integral del ciclo de vida de los usuarios, publicación de contenido multimedia con metadatos asociados y un motor de búsqueda indexado por hashtags.
- Técnico: Implementación de un ecosistema basado en Docker, automatización de despliegue mediante un pipeline de CI/CD y auditoría de calidad de código con SonarQube.
- Seguridad: Restricción de acceso a recursos mediante JSON Web Tokens (JWT) y generación de logs de auditoría para actividades administrativas.

15.2 Limitaciones

- Formatos de Archivo: El sistema procesará exclusivamente formatos de imagen estándar (JPG, PNG).
- Persistencia: No se permitirá el almacenamiento local de contenidos; toda la información será persistida en servicios de almacenamiento de objetos en la nube.
- Moderación: Aunque existen filtros automáticos, el estado final de las publicaciones en controversia dependerá de la validación manual del administrador.
- Conectividad: El sistema requiere de una conexión a red estable y constante debido a su arquitectura distribuida.